# Racket Macro: Match

```
(define-syntax (uncle-sams-holy-trinity stx)
#'(list Aws Luke Xiao))
```

# Examples of Match

```
(match '(1 2 3)
  [(list a b c)
   #:when (= 6 (+ a b c))
   'sum-is-six]
  [(list a b c) 'sum-is-not-six])
```

This example demonstrates many of the available features of match such as:

- **Variable Bindings** for **a b c**,
- **Optional Conditional Expressions #:when**,
- **Multiple Clauses.**

```
(match '(1 2)
  [(list a b c)
   #:when (= 6 (+ a b c))
   'sum-is-six]
  [(list a b)
   (match a
     [(? number?)
      'car-is-num])])
```

This example builds on the previous one by adding a **nested match**, which showcases the flexibility of the match macro.

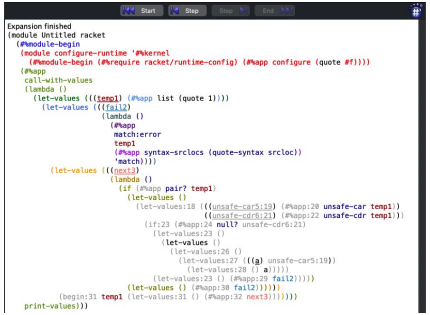
# Expanding Match

**match with variable binding**

```
#lang racket

(match (list 1)
  [(list a) a])
```

**macro stepper**




**match (expanded)**

```
(let [(temp1 (list 1))]
  (let [(next3
          (lambda ()
            (if (pair? temp1)
                (let [(unsafe-car5 (car temp1))
                      (unsafe-car6 (cdr temp1))]
                  (if (null? unsafe-car6)
                      (let [(a unsafe-car5)] a)
                      (error)))
                (error))))]
    (next3)))
```